Expose of the dissertation project

Working Title

Timing Analyzable Rust Compilation

Abstract

Every day, we trust our lives to cars, planes, and other complex machines. At their core, these systems are controlled by embedded computers that must react within strict timing bounds. An airbag controller, for instance must decide within a split-second on whether to inflate the airbag or not. Consequently, a system developer must verify that these timing bounds are met. Different tools exist for this purpose, but these require extensive configuration efforts and are cumbersome and error-prone. Currently, timing verification is an interactive process that requires manual annotations and expert knowledge from the system developer.

We will develop a self-contained toolchain, based around the programming language Rust, that will greatly simplify the timing verification process. Through the usage of Rust, we strive to be able to provide support for low-level compile targets and rigorous type-checks on compile time. We will extend the compile-time checks to verify the timing analyzability, implementing our paradigm "*if the program compiles, it is analyzable*".

We will support automatic generation of all inputs to the timing analysis, therefore eliminating the need for an *interactive* timing verification process. This will reduce both the effort and required knowledge about timing analysis from the side of the developer. It will also eliminate the need of running the expensive timing analysis tooling more than once.

Our *Timing Analyzable Rust Compilation* will reduce development time and development costs of embedded systems. It will enable timing verification without expert knowledge and replace an error-prone interactive process with a fully automated compiler-driven one. It thus will also increase the safety of embedded systems to which we trust our lives.

Key terms

Real-Time Systems, Embedded Systems, Programming language, Rust, Safety-Critical Systems

Introduction

The programming language Rust celebrates its tenth anniversary this year. Within these ten years, Rust has become a true alternative to other low-level programming languages such as *C* with added benefits such as memory safety, a strong type-system but also a modern tooling. In particular its security and safety features have raised interest such that it has even been mentioned and recommend explicitly by the U.S. White House. Roughly speaking, *if a Rust program compiles, it is safe*. Of course, safe only with respect to some, even though the most common, types of faults. Although the safety-checks can be overturned by the programmer by using the *unsafe* keyword, the general concept still stands, namely that successful compilation entails safety.

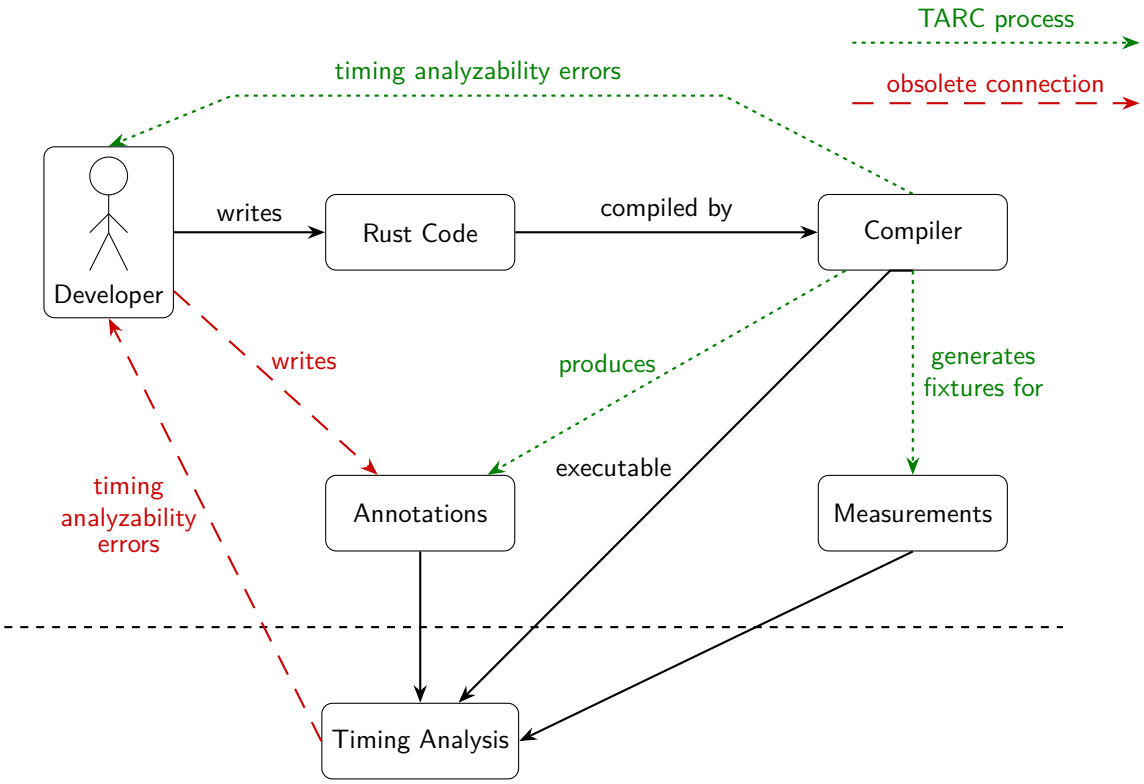


Figure 1: Overview of the current and foreseen development process of embedded real-time systems, including timing verification. Dashed lines represent the current workflow actions that we aim to eliminate, whereas the dotted lines represent novel action that we aim to establish (Timing Analyzable Rust Compiler or *TARC* process). Instead of using the current feed-back loop from Rust-code to compiler and timing analysis back to the programmer, we want to make sure that once the Rust program compiles, the timing analysis will be successful and will not require further input from the programmer.

Within this PhD thesis, we want to translate this principle to embedded real-time systems. In embedded real-time systems, next to safety and security, we must be able to provide timing guarantees for at least some of the code that we run on our embedded systems. Think for instance of an

automatic emergency breaking system that must not only make the right call, but also make this within a guaranteed timing bound. So far, providing timing guarantees is a cumbersome and overly error-prone process that requires in depth knowledge and effort from the system developer. All loops must be bounded, memory accesses must be known, pointers must be determined statically and so on. This information is then given to the worst-case execution time analysis, which uses it to derive a bound on the worst-case execution time. There are several competing static timing analysis tools, some measurement-based tools and even some in-house developed timing analysis tools are in use in the industry. For all of them, static information about the program execution time is highly valued but difficult to achieve.

Our aim is to ensure that *if a Rust program compiles, it is analyzable*. Even more so, we want to extend the Rust-compiler so that it automatically provides all annotations that are needed for a successful timing analysis. Compilation will entail timing-analyzability without further user-interactions and the availability of all static information needed to conduct a worst-case execution time analysis.

The paradigm-shift is visualized in Figure 1. It shows the actions that will made obsolete (like the need for the developer to write timing-analysis-specific annotations), the ones that are repositioned (like the feedback through timing analyzability errors, that are now provided by the compiler in order to speed up the feedback loop) and finally the ones that are added (as per the new design the annotations are directly produced by the compiler and the suggested architecture in theory supports automatic generation of measurement-fixtures). The visualization also shows the separation that allows us to treat the timing analysis as a *black box*. In the currently established processes, the developer needs to react to specific errors, that are thrown by the timing analysis after the manual annotation process. With our newly proposed process, the errors are thrown by the compiler, while the code is written – this prevents non-analyzable code before the timing analysis tries to analyze it in vain. Furthermore, this removes the developers direct interactions with the timing analysis, eliminating the need for the developer to be accustomed to details of the timing analysis tools.

Rust is the optimal choice for such an extension to validate timing analyzability. Its strict typing and borrowing rules eliminate many of the problems of analyzing C-Code. Rust provides a modern and extensible compiler toolchain that already implements several of the static analyses required to check timing analyzability. We will build upon these tools and analyses for our Timing Analyzable Rust Compilation.

Following this statement, we present a mock-up of the planned analyzable-Rust syntax in Listing 1. The only requirement for the toolchain to work, is the marking of the analysis-entrypoint with the **ta_analyzable** macro.

```rust
use analyzable_macro_attribute::{ta_analyzable, ta_bounded};

fn main() {
    check_airbag_trigger_threshold();
}

const NUM_SENSORS: usize = 6;
const THRESHOLD: usize = 2000;

#[ta_analyzable()]
fn check_airbag_trigger_threshold() -> usize {
    let mut sensor_values = vec![0usize; NUM_SENSORS];

    for i in 0..NUM_SENSORS {
        ...
    }

}
```

Listing 1: Use of the **ta_analyzable** macro in Line 10. This macro both creates a entry point for the timing analysis and forces the compiler to validate the timing analyzability of the following program code.

Similarly to the unsafe-region in standard Rust, an additional macro can be used within the timing analyzable region to exclude this part from timing analyses and instead replace it by an upper bound provided by the programmer. The syntactical details are shown by Figure 2.

```rust
use analyzable_macro_attribute::{ta_analyzable, ta_bounded};

...

    for i in 0..NUM_SENSORS {
        sensor_values[i] = ta_bounded!({ read_sensor_value(i) }, 45);
        ...
    }

...
```

Listing 2: By using the **ta_bounded** macro in Line 6. The call to *read_sensor_value(i)* will not be analyzed. Instead, this call will be annotated by its upper bound of 45 cycles.

The proposed functionality is planned to be implemented by the *procedural macro* support, that is build directly into the Rust language, as described by the *Rust Book* [1]. The syntax in these examples is not final, but the conceptual design will remain the same.

Research question

Our research question is formulated as follows: Can we add timing analyzability as a first-class citizen of Rust and introduce the paradigm: **if a Rust program compiles, it is analyzable**.
To this end, we will explore whether it is possible to automatically extract a comprehensive set of flow-fact and memory annotations that will enable a timing analysis without any further interaction or input by the programmer. The Rust programming language does not per-se disallow unsafe regions, inline assembly or even infinite loops. We hence will investigate the maximum subset of Rust that still allows for automatic timing analyzability. Ultimately, we want to enable Rust-Programmers to write analyzable Rust programs and subsequently perform a timing analysis without any further knowledge about timing analyses.
We will evaluate the success of the project on a recent Rust-Port [2] of TACLeBench [3][1], TACLeBench represents the de-facto standard benchmark suite for embedded real-time systems [4]. We will deem the research project a success, if we are able to analyze at least 75% of these benchmarks without any changes to the Rust-Code. In addition, we will apply our Timing Analyzable Rust Compilation on a simple Smart-Home application implemented in Rust [5] to validate the toolchain on a complete system implementation.

Research status

Timing Verification of Safety-Critical Systems

Timing analyses, i.e. analyses that bound a program's worst-case execution time [6] (WCET), can be classified into static [7, 8], measurement-based [9] and probabilistic [10, 11], with some hybrid solution in between [12]. There are also a limited number of commercial timing analyses, foremost from Absint[2] and Rapita[3], although a recent industrial survey [13] has shown that a majority of real-time companies use their own, in-house developed timing analyses. Most of these tools resort to the binary level and measure execution times of programs or program fragments (measurement-based analysis) or employ complex processor models (static analysis).
Static timing analysis consists of several intermediate steps. First, the analysis reconstructs the control-flow graph either from binary or from source-code. Next, a control-flow analysis determines loop bounds, identifies all possible execution paths and derives the effective address for each memory access. In the case of cached architectures, a cache analysis is required to classify the memory accesses as cache-hits and cache-misses. A hardware model then provides execution time bounds for all basic blocks. The last step of the static timing analysis combines the control-flow information, the loop bounds and the execution time bounds of the blocks to determine the longest execution path through the program. To this end, the analysis formulates an integer linear program that implicitly models the feasible control-flow. This technique is called implicit path enumeration, or IPET. In case of a hybrid analysis, the execution times of basic blocks are measured instead of analyzed, whereas the steps (control-flow analysis and implicit path enumeration) remain the same. Completely measurement-based approaches often rely on end-to-end measurements of the programs and comprehensive test regimes.

[1] `https://github.com/tacle/tacle-bench`
[2] `https://www.absint.com/ait/index.htm`
[3] `https://www.rapitasystems.com/wcet-tools`

Annotations required to perform a static or hybrid timing analysis typically fall into one of the three categories:

Control Flow Annotations A timing analysis must be able to identify all execution paths. While simple program structures do not pose a challenge to the timing analysis, function pointers for callback functions and similar constructs can typically not be resolved statically and may prevent timing analysis in the first place. Infeasible paths on the other hand may increase the pessimism of the WCET bounds and vice-versa. Excluding such paths from the analysis leads to tighter timing bounds. Control Flow Annotations thus resolve the function addresses or provide information about infeasible paths.

Loop Bound Annotations The worst-case execution time is only finite, if all loops are bounded. Static loop-bound analyses are able to determine loop bounds for common loop structures but standard compiler optimization may already prevent a loop-bound analysis. Nested loops and complex, input-dependent loop exit conditions pose further challenges that require manual annotations of loop bounds.

Memory Access Annotations The effective address of a memory access can have a tremendous impact on the timing behavior of the access. An unaligned access may trigger fault-handling and cause additional accesses compared to an aligned access. The memory access time may depend on the accessed memory region in case of NUMA-architectures. And last but not least, static cache analyses require exact knowledge about the effective memory address of an access to provide classification of the memory access.

Purely measurement-based timing analyses do not require annotations, like they are needed for static timing analysis. Nevertheless, these annotations are still helpful in order to guide test-cases and to reduce the number of tests required for the analysis.

There are a few approaches that integrate timing analysis within the C-Compiler. Foremost, the WCET-aware C Compiler WCC [14] and LLVMTA [15]. WCC aims at optimizing the worst-case path through the program, i.e. reducing a program's WCET bound, while LLVMTA is more of a test-environment for different types of micro-architectural analyses. Both compilers still rely on user annotations and do not force the programmer to write analyzable code.

In stark contrast to prior work, we aim to focus on the Rust-programming language. Rust has only received limited attention in safety-critical real-time domain so far, and as far as we know, it has not received any attention with respect to the timing analyzability. Its paradigm "if it compiles, it is safe" provides a novel opportunity to lift the timing analysis challenge from binary- to the compiler-level. Exploiting its extensive error-reporting and the available static analyses, our extended Rust-compiler will automatically generate a comprehensive set of annotations, or indicate which program part cannot be analyzed.

The low-level timing analysis will not be part of the research proposal, but we will instead use existing tooling for this purpose. Yet, with the complete set of annotations, the timing analysis tool will basically become a one-click tool that does not require any user-interaction anymore.

Current Coding Guidelines and Best Practices for C

The programming language C is notoriously error-prone and difficult to analyze. Coding guidelines such as MISRA-C [16], DO-178C [17] and NASAs Rule of Ten [18] compensate for some of the shortcomings of C by imposing strict requirements on the coding style. Yet, the guidelines are imposed on top of the language and not integrated within. External, commercial tools[4] may check for compliancy but create an additional indirection. Furthermore, following these guidelines improves analyzability, but does not guarantee that the programs following these standards are analyzable: Even following best practices from the Security-Domain does not entail timing analyzability, as shown in [19].

Research on Rust for Embedded/Real-Time Systems

Although the Rust programming language was originally perceived to build safe, concurrent and fast browser engines, it has since been adopted in many domains. In particular its unique combination of memory/type/concurrency-safety and execution speed make Rust a perfect choice for the development of safety-critical and embedded systems. At the same time, these domains are known for slow adoption of novel technologies due to certification requirements and generally conservative mindsets. Notable initiatives to use Rust for Embedded Systems include Safety-Critical Rust[5] and MISRA-2025[6] that aims to include Rust. As far as we are aware, none of these initiatives specifically target real-time aspects or the timing analyzability of Rust programs. We will nevertheless consider the safety-critical subset of Rust as a starting point for our research.

From the perspective of Rust over C

In the previous section we have stated our strategy to replace the established C-programming language with the newer and more modern language Rust. Through the use of examples, the following section is supposed to substantiate our claims for this choice.
One of the greatest problems of C in terms of the ability to reliably determine data-types is the *pointer-arithmetic* that is typically used to drill into structures.

[4] `https://www.absint.com/releasenotes/astree/14.10/index.htm`
[5] `https://rustfoundation.org/safety-critical-rust-consortium/`
[6] `https://www.parasoft.com/blog/misra-c-2025-rust-challenges/`

```c
typedef struct
{
    int x;
} Small;

int main()
{
    Small arr[3] = {{1}, {2}, {3}};

    Small *ptr = (Small *)((char *)arr + sizeof(Small));

    printf("Second element: %d\n", ptr->x); // prints 2

    return 0;
}
```

Listing 3: C-Example of pointer arithmetic that poses a challenge to the timing analysis

The fact that no definite memory-size of the structure is stored alongside the pointer makes it very hard to run cache-analyses for more complicated structures. Rust does not have this problem, as direct memory access via pointer arithmetic is forbidden outside of unsafe regions. Pointers in Rust contain additional metadata, a concept known as *smart pointers* [20], requiring them to hold information about *ownership* of that data or the aforementioned struct-size.

```rust
#[derive(Debug)]
struct Small {
    x: i32,
}

fn main() {
    let arr = [Small { x: 1 }, Small { x: 2 }, Small { x: 3 }];

    let second = &arr[1];

    println!("Second element: {}", second.x); // prints 2
}
```

Listing 4: Example of smart pointer as used in Rust that include meta-data on ownership and struct-sizes

The fact that in C pointers hold no metadata, shifts the responsibility onto the programmer. Many functions that must be able to deal with different structures of shared functionality solve this by implementing callbacks to take *void* pointers.

```c
void *print_number(void *arg)
{
    int *num_ptr = (int *)arg;

    printf("Number: %d\n", *num_ptr);

    return NULL;
}

int main()
{
    pthread_t thread;
    int number = 42;

    // Pass pointer to number as void*
    if (pthread_create(&thread, NULL, print_number, (void *)&number) != 0)
    {
        perror("Failed to create thread");
        return 1;
    }

    pthread_join(thread, NULL);
    return 0;
}
```

Listing 5: Use of Void-Pointer in C

This concept however provides absolutely no assurance that these functions are only called with proper arguments. Writing such functions is impossible in Rust. For shared functionality, Rust uses the concept of *traits* [21], which allows for much greater control about the passed data during compile-time.

When C is called out as an *unsafe* language, this boils down to buffer overflows and other kind of unwanted memory access most of the time. The following example in C shows an unprotected write to an array. Writing a name of length five works as expected, however writing a name of length ten into the array will result in the secret being modified without the program crashing.

```c
typedef struct
{
    char name[8];
    int secret_code;
} User;

int main()
{
    User u;

    u.secret_code = 123456;

    printf("Enter your name: ");
    gets(u.name); // gets() des not check bounds

    printf("Hello, %s!\n", u.name);
    printf("Secret code is: %d\n", u.secret_code);

    return 0;
}
```

Listing 6: C-Example of unprotected write to C

The last example leads to an important property of Rust that needs to be pointed out. It is *not* impossible to crash a Rust program, nor is it impossible to run into bugs or security vulnerabilities related to errors in the sequence-logic of the program. While the following program looks correct on the first glance, it will terminate on runtime, because the parse-function expects the numbers to contain dots and no commas.

```rust
fn main() {
    let input = "42,2";

    let num: f32 = input.parse().unwrap();

    // go on with an important calculation

    println!("Parsed number: {}", num);
}
```

Listing 7: Unwrapping in Rust to prevent undefined behavior that poses a common problem in C

The important difference is, that such crashes are always tied to a *panic* event (in this case caused by the *unwrap* call). With the language actively panicking, Rust will never exhibit *unexpected behavior*. This greatly narrows down the paths that an assembly program could take, as many branches are actively cut. Secondly, it is easier to analyze if a program will crash, as functions can be statically scanned for panic actions. For the determination of the suggested maximal analyzable Rust-subset, it will be necessary to decide if error handling will need to be enforced, or how a panic is going to be treated.

As the last example, a classical *use after free* is shown in C and Rust.

```c
int main()
{
    int *ptr = malloc(sizeof(int));
    *ptr = 42;

    free(ptr);
    printf("%d\n", *ptr); // USE AFTER FREE! Undefined behavior

    return 0;
}
```

Listing 8: Flawed C-Example that will lead to a runtime error

```rust
fn main() {
    let ptr = Box::new(42);

    drop(ptr); // explicitly free memory

    println!("{}", *ptr);
}
```

Listing 9: Flawed Rust-Example that will lead to a compiler-time error

Comparing this to the Rust-version of the code, at first it might not be obvious what the difference is. It however becomes apparent, when the programs are compiled. The C-version produces exceptions only on *runtime*. The Rust-compiler throws its exceptions already at *compile-time*. This important difference should be carried into the timing analysis. We do not only want to provide programs to the timing analysis and see what the results are, but by imposing rules during the compilation-step, we want to ensure that only analyzable programs are ever handed to the external timing analysis.
Finally, it should be noted that Rust has native features to ensure consistency across threads. As the research in this thesis is limited to single-core targets, this will be not of immediate benefit but helpful for future research.

Personal experience and prior knowledge

The topic of Rust can be viewed as the hook to bring me and he chair of embedded systems together to consider working on this research suggestion. As it can be seen on my GitHub page[7], I have worked on multiple smaller projects, written in the Rust language, in my personal time. When it comes to different types of programming languages, I have always preferred strongly-typed and compiled ones to scripting-languages.
My biggest Rust-project to date is the *Mathezirkel-App* [22], a program to manage the personal data of students in relation to the "Mathezirkel der Universität Augsburg". This project is in-progress, but

[7] https://github.com/jonas-kell

is being actively used for several years. So far over 700 hours have been spent on this project, with a large portion of that spent on the backend, which is written in Rust from the ground up.
Furthermore, in the context of my master thesis I have written an application to handle mathematical statements analytically, called "Math Manipulator" [23]. The project was used as an exercise for custom lexing, parsing and handling of operator trees, but later grew to have direct application to the results of my master thesis. This experience will be invaluable for the implementation of the macro to be able to parse a large subset of Rust's syntax reliably.

The chair for Embedded Systems at the University of Augsburg has ample experience with real-time systems in general and timing analysis in particular. Precisely on the topic of the research project, the group has published in 2024 a paper on timing analyzability of cryptographic implementations [19]. Prior work of the chair also includes the development of timing analyses for multi-core systems [24, 25], predictable system design [26, 27, 28] and timing- and cache-analyses [29, 30, 31].
The chair for Embedded Systems has recently started to look into Rust for embedded systems. This includes bachelor [2] and master [5] theses and incorporation of Rust within the course program.

Methodology and outline

It is typically not necessary to provide timing guarantees for the entire executable. We will thus introduce a macro **ta_analyzable** that indicates:

- Here is a new entry point for the timing analysis, i.e., we need to derive a timing bound for the subsequent scope.
- The following scope must be analyzable and all annotations to perform a timing analysis must be derived.

This allows for a structured analysis, that has a clearly defined scope and offers the possibility to inject configuration and extra information precisely at the relevant locations. This way of defining the analyzable regions serves to tie the parameters, that are handed to the analysis-process, to the respective code. Through this design choice, it should be easier to inspect and modify the code, as the necessary metadata can be stored alongside the source code and not in an external configuration.
An additional macro **ta_bounded** within a ta_analyzable-scope will have the opposite effect. A ta_bounded-scope will not be analyzed as rigorously by the Rust-toolchain, similar to an unsafe-region, but will be fed to the timing analysis nevertheless. This is necessary for calls of external systems or complex functionality that is not directly integratable into automatic analysis. An external system call might have a defined maximum response time that can however not be derived from the program alone. This also makes it possible to use syntactical constructs that can be analyzed manually, but surpass the capabilities of the automatic analysis we aim to provide.
To be able to integrate these regions into the timing analysis, it will be possible to optionally provide a manually calculated timing bound to be used in the overall calculations. Such a bound could originate from hardware specifications, external tools, manual runtime calculations or measurements.
Summarizing this plan, this should result in a system that allows a Rust programmer to be able to perform a timing analysis simply be using these macros. This entails the advantage, that modifica-

tions to the code base can be made or even completely new subsystems can be designed, without specific knowledge on timing analysis being a definite requirement.
Concerning measurement-based timing, the Rust-macro [1] approach also plays an important role. While the applications, as described so far, only require passive analysis and reporting on compile time – interrupting the compile process to give feedback and exporting annotations for external tooling both works without modification of the source code – the typical usage of macros lies in the active replacement or changes to parts of the source code. In that domain, Rust has a much more powerful macro-system in place than most other languages. The language allows for re-writing of its own code on a token level, inside the language with access to basically all language features.
With that level of control, we will be able to inject measurement-fixtures at arbitrary points throughout the analyzed code, making it possible to compile for live usage and simultaneously for performing a measurement-based timing analysis. The macro might thereby inject different timing-code, based on the target architecture and timing capabilities. All these features can be obtained from one version of the source code, without manual introduction of timing-related code or performance overhead in the live application.
It is even thinkable to introduce timing code piece-wise: As some time-measurements could even be done fully automated by the toolchain, the system might compile several versions of the code – each introducing only the bare set of test-fixtures for a specific measurement – to minimize the influence of measurements in comparison to the final runtime. As this in of itself could be both be a wanted or unwanted consequence, the flexible fashion of this approach allows for precise control over the behavior of the code introduced at compile time.
To adhere to the plan of treating timing analysis as a black box for the research during this thesis, measurement-based analysis will not be investigated extensively. Still, as it is taken into consideration from the beginning of planning, future, related work will be able to integrate this as a natural extension to our toolchain.

Definition of work packages

To define the outline for the execution of the thesis, we bundle smaller parts into five *work packages* that will be named and explained in detail in the following section. The schedule will be presented in the section following this listing and embed these same work packages in a time-plan for the duration of the project.

1. **Implement the macro-infrastructure within Rust:**
   This is a basic implementation task for the start of the research. This topic includes the writing of a robust and expandable infrastructure that can take care of all the technical requirements. For this purpose, the detailed behavior of the compiler needs to be studied and the underlying syntax for the macro needs to be pinned. It also needs to be checked, that the technical integration of external software for timing analysis purposes can be done cleanly. Finally, general interfaces between the systems need to be established.

2. **Enumerate all possible annotations:**
   To allow for static timing analysis of a compilable Rust-program, the outline of what can and needs to be analyzed must be defined. This includes a rigorous listing of what possible

memory accesses could be taken by the compiler to retrieve the data and map the data-structures that could pop up to the corresponding way of access. We need to define possible cache-architectures of the target system and write a memory-utilization-simulation to quantify the cache timings. To interact with external analysis software, this step requires a matching of all available and needed annotations between the two systems.

3. **Implement the static analysis for a minimal subset of Rust:**
   This requires translating all the high-level concepts of Rust to the annotations that the timing analysis tool needs to work. While this needs to be able to treat all the concepts of a high-level language, it will be still very restrictive in terms of what syntax and chaining of operations is allowed to be used.

4. **Derive the maximal timing analyzable subset of Rust:**
   This can be done iteratively, integrating and checking more and more of the available syntax and combinations of syntax. We thereby start with the *minimal* syntax to write a program, then expand this to include more and more syntactical and convenience concepts. While for example in the beginning it would only be allowed for a loop bound to be stored in a constant, later iterations might allow syntactical sugar, like iteration up to the length of a constant field or array, without extracting it to a separate constant first. This is explained in detail at the end of this section.

5. **Evaluate Analyzable-Rust:** To put the compiler to a test, we search for existing projects to be analyzed, or make up solutions to real-life code. As stated earlier, we aim to be able to analyze a large portion of TACLeBench [3] with the newly derived tools. Furthermore, as the number of important projects that are re-written in Rust is recently starting to increase, it is likely that until we reach this point in the research, additional examples might have already been ported to or been freshly released in Rust.

In Listing 10 we again want to present an example function for the use of the macros. According to the work packages, the following flow is intended for analyzing such a code snippet.
First, the compilation should only allow analyzable constructs. An example for this case would be to forbid *infinite loops (with break)* expressions. Consequently such a construct is not found in this snippet.
As a second step, the static analysis would be ran. An example for this can be seen in line 14, where the loop bounds and other relevant metadata can be determined from the constant values. For this purpose, we will not develop new analyses, but select appropriate ones from the wide selection of available research [32, 33, 34].
The information given to the *ta_bounded* block in line 18 is taken into account for the static analysis. Finally, we plan to expand the analysis to include more complicated syntax, like the for-in loop in line 22. To determine the loop bound for this case, a more sophisticated process is required, as the information about the number of elements in the vector can only be obtained from earlier lines. Modern compilers also already include a wide range of program analyses that we will exploit [35].

```rust
#[ta_analyzable()]
fn check_airbag_trigger_threshold() -> usize {
    let mut sensor_values = vec![0usize; NUM_SENSORS];

    for i in 0..NUM_SENSORS {
        // first argument in ta_bounded is the code to insert,
        // second one is a maximum runtime from an external source
        // (measurement, analysis, calculation etc.)
        sensor_values[i] = ta_bounded!({ read_sensor_value(i) }, 45);
    }

    let mut count = 0usize;
    for &value in &sensor_values {
        if value > THRESHOLD {
            count += 1;
        }
    }

    count
}
```

Listing 10: The complete function that demonstrates the functionality of the macros **ta_analyzable** and **ta_bounded** from the listings 1 and 2.

Generally, giving an exact timing analysis for an arbitrary program requires the solution of the *halting problem*, which is famously proven to be impossible so solve. Because of this, the *maximum* in the case of the *maximal timing analyzable subset of Rust* (see work package 4) does not refer to the strict mathematical definition of the precise greatest number of operator-combinations that can be really analyzed, but rather refers to the greatest number of programming *concepts* that can be defined strict enough to be both useful and possible to work with – with "useful" in this case meaning that we do not want to diminish the power of the Rust-programming language by excluding core concepts. Conveniently, the *possibility* to work with a concept includes the time-frame that is required to solve its timing analyzability.

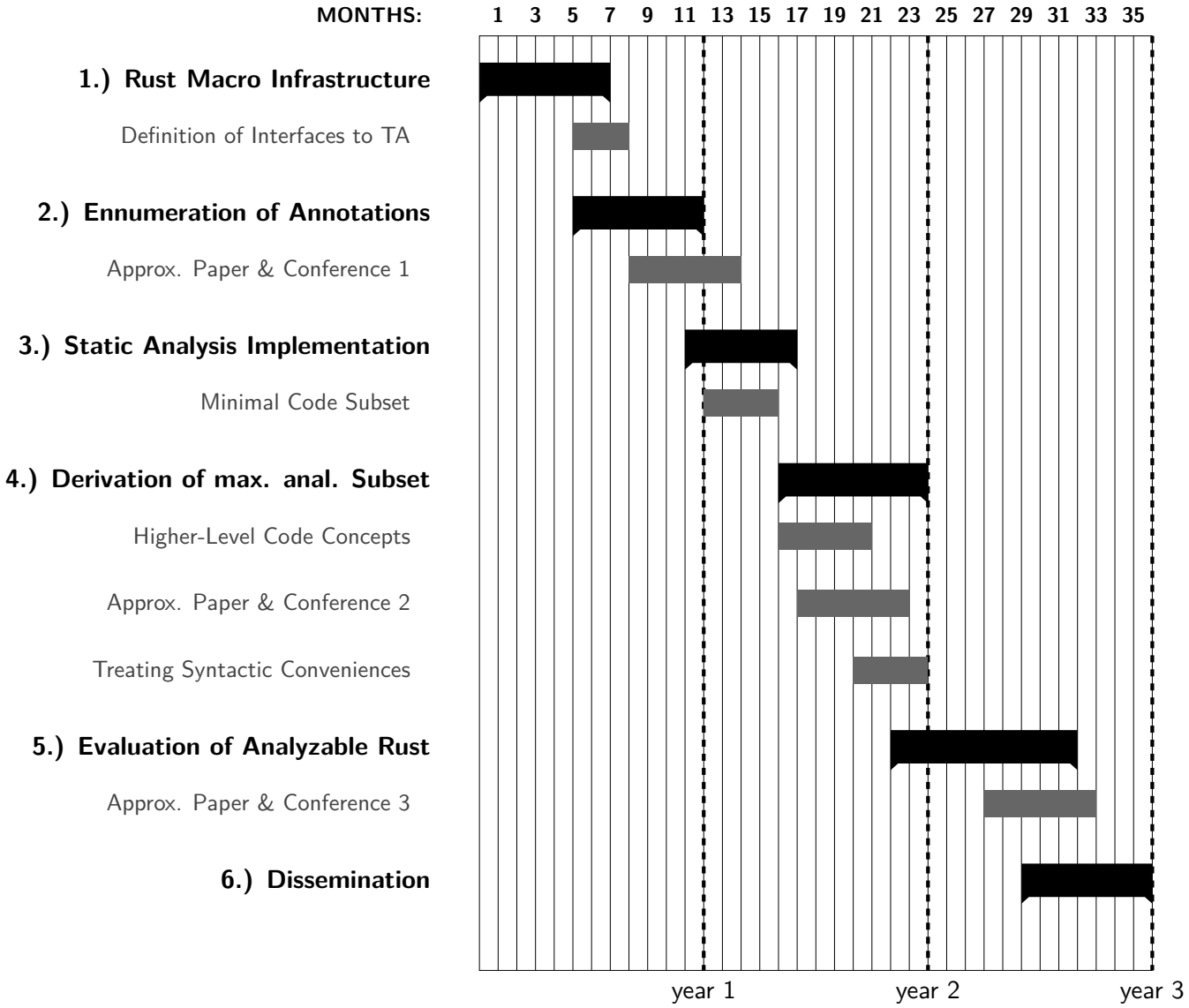
To explain this more practically: As a research goal it would be best, to be able to analyze *all* possible programs. But this is proven to be an impossible goal. Therefore, the next best thing is to attempt to make *as many programs as possible* analyzable – which we plan to do in an iterative process. For a program to be *analyzable*, it must not contain unbounded loops. The challenge lies in ensuring this property.

The most simple programs can be analyzed trivially, but that analysis is probably not useful for any application. Taking more and more programming concepts into account one after another, gives a control nob for the total duration of the project. Depending on the amount of time that the higher level concepts are going to take to describe, more or less code concepts and or syntactic conveniences can be integrated in the subset of analyzable expression-combinations.

This leads us to the design of a schedule for the work on the thesis.

Planned schedule

Typically, giving a precise and tight time estimation for a project with dissertation level complexity is not an easy task, as unforeseen roadblocks in the research might multiply the time that is taken for any specific step. However, due to the design of the research question, this project can be tackled in a flexible fashion, to guarantee that a completed work can be reached during the given time-frame of three years.
The main contribution to this flexibility is provided by the work package number four, which is the *derivation of the maximal timing analyzable subset of Rust*. As explained earlier, the complexity of the work of this package can be intensified or relaxed, based on the available time, without compromising achieveability or quality of the overarching goal. In this case, the chosen strategy still allows for targeting an exact duration of 36 months, without diminishing the quality of any major goals. This will allow to obtain a completed result exactly in the given time-frame.
As it is customary in the field of embedded systems, the research is often tied closely to publishing papers for and visitation of conferences and workshops. The plan includes the attendance of three of such events, including preparation and writing, as well as the travel time itself. For any of these alone, we calculate an average time expenditure of three months, totaling a combined time of nine months. These will need to be situated in the schedule dynamically, based on the acceptance into peer-reviewed publishing and the dates of future events. Because of this, the time that is allocated for each of the (sub-) tasks is longer, to include these nine months spread evenly over all available tasks.
This also reflects the true nature of the work more closely, as generally the work on the defined work packages and the publishing of intermediate results will happen in parallel.
Still, it is important to remember this fact when arguing about the the time-allocation of the work packages, as without the publishing the time-table would target a duration of 25 months instead of 36. Holidays and free time are thereby also assumed to be spread evenly across all the working time, to make the calculation simpler.
To be able to best present the relations between the work packages, as well as the amount of time we plan to allocate for their completion, the schedule was chosen to be displayed with the following chart:



Literature overview

[1] *Macros - The Rust Programming Language*. URL: `https://doc.rust-lang.org/book/ch20-05-macros.html` (visited on 07/14/2025).

[2] David Heim. "Analyse der Echtzeitfähigkeit von Rust". German. Betreuer: Johannes Kühbacher. Masterarbeit. Universität Augsburg, May 2025.

[3] Heiko Falk et al. "TACLeBench: A Benchmark Collection to Support Worst-Case Execution Time Research". In: *16th International Workshop on Worst-Case Execution Time Analysis (WCET 2016)*. Ed. by Martin Schoeberl. Vol. 55. Open Access Series in Informatics (OASIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2016, 2:1–2:10. ISBN: 978-3-95977-025-5. DOI: `10.4230/OASIcs.WCET.2016.2`. URL: `https://drops.dagstuhl.de/entities/document/10.4230/OASIcs.WCET.2016.2`.

[4] Tilmann L. Unte and Sebastian Altmeyer. "Real-Time System Evaluation Techniques: A Systematic Mapping Study". In: *37th Euromicro Conference on Real-Time Systems (ECRTS 2025)*. Ed. by Renato Mancuso. Vol. 335. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 12:1–12:21. ISBN: 978-3-95977-377-5. DOI: 10.4230/LIPIcs.ECRTS.2025.12. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECRTS.2025.12.

[5] Julian Hoffmann. *Rust als Alternative zu C++ in Embedded Systems: Eine vergleichende Analyse*. German. Bachelorarbeit. Gutachter: Johannes Kühbacher. 2025.

[6] Reinhard Wilhelm et al. "The worst-case execution-time problem—overview of methods and survey of tools". In: *ACM Trans. Embed. Comput. Syst.* 7.3 (May 2008). ISSN: 1539-9087. DOI: 10.1145/1347375.1347389. URL: https://doi.org/10.1145/1347375.1347389.

[7] Reinhold Heckmann and Christian Ferdinand. "aiT: Worst-Case Execution Time Prediction by Static Program Analysis". In: *International Federation for Information Processing Digital Library; Building the Information Society;* 156 (Jan. 2004). DOI: 10.1109/IPDPS.2004.1303088.

[8] Damien Hardy, Benjamin Rouxel, and Isabelle Puaut. "The Heptane Static Worst-Case Execution Time Estimation Tool". In: *Worst-Case Execution Time Analysis*. 2017. URL: https://api.semanticscholar.org/CorpusID:20377524.

[9] I. Wenzel et al. "Measurement-based worst-case execution time analysis". In: *Third IEEE Workshop on Software Technologies for Future Embedded and Ubiquitous Systems (SEUS'05)*. 2005, pp. 7–10. DOI: 10.1109/SEUS.2005.12.

[10] Francisco J. Cazorla et al. "Probabilistic Worst-Case Timing Analysis: Taxonomy and Comprehensive Survey". In: *ACM Comput. Surv.* 52.1 (Feb. 2019). ISSN: 0360-0300. DOI: 10.1145/3301283. URL: https://doi.org/10.1145/3301283.

[11] Liliana Cucu-Grosjean et al. "Measurement-Based Probabilistic Timing Analysis for Multi-path Programs". In: *Proceedings of the 2012 24th Euromicro Conference on Real-Time Systems*. ECRTS '12. USA: IEEE Computer Society, 2012, pp. 91–101. ISBN: 9780769547398. DOI: 10.1109/ECRTS.2012.31. URL: https://doi.org/10.1109/ECRTS.2012.31.

[12] Daniel Kästner et al. "TimeWeaver: A Tool for Hybrid Worst-Case Execution Time Analysis". In: *19th International Workshop on Worst-Case Execution Time Analysis (WCET 2019)*. Ed. by Sebastian Altmeyer. Vol. 72. Open Access Series in Informatics (OASIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, 1:1–1:11. ISBN: 978-3-95977-118-4. DOI: 10.4230/OASIcs.WCET.2019.1. URL: https://drops.dagstuhl.de/entities/document/10.4230/OASIcs.WCET.2019.1.

[13] Benny Akesson et al. "A comprehensive survey of industry practice in real-time systems". In: *Real-Time Syst.* 58.3 (Sept. 2022), pp. 358–398. ISSN: 0922-6443. DOI: 10.1007/s11241-021-09376-1. URL: https://doi.org/10.1007/s11241-021-09376-1.

[14] Heiko Falk, Paul Lokuciejewski, and Henrik Theiling. "Design of a WCET-Aware C Compiler". In: *2006 IEEE/ACM/IFIP Workshop on Embedded Systems for Real Time Multimedia*. 2006, pp. 121–126. DOI: 10.1109/ESTMED.2006.321284.

[15] Sebastian Hahn et al. "LLVMTA: An LLVM-Based WCET Analysis Tool". English. In: *20th International Workshop on Worst-Case Execution Time Analysis*. Ed. by Clement Ballabriga. Open Access Series in Informatics (OASIcs). Schloss Dagstuhl, July 2022, 2:1–2:17. DOI: 10.4230/OASIcs.WCET.2022.2.

[16] Roberto Bagnara, Abramo Bagnara, and Patricia Hill. "The MISRA C Coding Standard and its Role in the Development and Analysis of Safety- and Security-Critical Embedded Software: 25th International Symposium, SAS 2018, Freiburg, Germany, August 29–31, 2018, Proceedings". In: Aug. 2018, pp. 5–23. ISBN: 978-3-319-99724-7. DOI: 10.1007/978-3-319-99725-4_2.

[17] Benjamin Brosgol. "Do-178c: the next avionics safety standard". In: *Proceedings of the 2011 ACM Annual International Conference on Special Interest Group on the Ada Programming Language*. SIGAda '11. Denver, Colorado, USA: Association for Computing Machinery, 2011, pp. 5–6. ISBN: 9781450310284. DOI: 10.1145/2070337.2070341. URL: https://doi.org/10.1145/2070337.2070341.

[18] G.J. Holzmann. "The power of 10: rules for developing safety-critical code". In: *Computer* 39.6 (2006), pp. 95–99. DOI: 10.1109/MC.2006.212.

[19] Alexander Stegmeier et al. "Safe and Secure? On the Timing Analysability of Cryptographic Implementations". In: *2024 IEEE 30th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 2024, pp. 68–80. DOI: 10.1109/RTAS61025.2024.00014.

[20] *Smart Pointers - The Rust Programming Language*. URL: https://doc.rust-lang.org/book/ch15-00-smart-pointers.html (visited on 07/14/2025).

[21] *Generic Types, Traits, and Lifetimes - The Rust Programming Language*. URL: https://doc.rust-lang.org/book/ch10-00-generics.html (visited on 07/14/2025).

[22] Jonas Kell. *Mathezirkel App*. 2023. URL: https://gitlab.com/mathezirkel/mathezirkel-app/backend.

[23] Jonas Kell. *Math-Manipulator Tool - Info and Repository*. 2024. URL: https://github.com/jonas-kell/math-manipulator.

[24] Axel Wiedemann, Florian Haas, and Sebastian Altmeyer. "Manatee: a multicore interference analysis tool for embedded soc evaluation". In: *Computer Science and Information Systems* 22.2 (2025), pp. 591–621. DOI: 10.2298/csis240724024w.

[25] Claire Maiza et al. "A survey of timing verification techniques for multi-core real-time systems". In: *ACM Computing Surveys* 52.3 (2019), pp. 1–38. DOI: 10.1145/3323212.

[26] Sebastian Altmeyer, Reinder J. Bril, and Paolo Gai. "EMPRESS: an efficient and effective method for PREdictable stack sharing". In: *2018 IEEE 24th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA), 28-31 August 2018, Hakodate, Japan*. Ed. by Rodolfo Pellizzoni, Insik Shin, and Kaori Fujinami. 2018. ISBN: 9781538677599. DOI: 10.1109/rtcsa.2018.00020.

[27] Jan Reineke et al. "Selfish-LRU: preemption-aware caching for predictability and performance". In: *2014 IEEE 19th Real-Time and Embedded Technology and Applications Symposium (RTAS), 15-17 April 2014, Berlin, Germany*. Ed. by Richard West, James Anderson, and Samarjit Chakraborty. 2014. ISBN: 9781479948291. DOI: 10.1109/rtas.2014.6925997.

[28] Sebastian Altmeyer et al. "On the effectiveness of cache partitioning in hard real-time systems". In: *Real-Time Systems* 52.5 (2016), pp. 598–643. DOI: 10.1007/s11241-015-9246-8.

[29] Boudewijn Braams, Sebastian Altmeyer, and Andy D. Pimentel. "EDiFy: an execution time distribution finder". In: *Proceedings of the 54th Annual Design Automation Conference 2017 - DAC '17, June 2017, Austin, TX, USA*. 2017. ISBN: 9781450349277. DOI: 10.1145/3061639.3062233.

[30] Sebastian Altmeyer, Claire Maiza, and Jan Reineke. "Resilience analysis: tightening the CRPD bound for set-associative caches". In: *ACM SIGPLAN Notices* 45.4 (2010), pp. 153–162. DOI: 10.1145/1755951.1755911.

[31] Sebastian Altmeyer et al. "Parametric timing analysis for complex architectures". In: *2008 14th IEEE International Conference on Embedded and Real-Time Computing Systems and Applications, 25-27 August 2008, Kaohsiung, Taiwan*. Ed. by Chi-Sheng Shi, Gerhard Fohler, and Ichiro Satoh. 2008. ISBN: 9780769533490. DOI: 10.1109/rtcsa.2008.7.

[32] Arnaud Venet and Guillaume Brat. "Precise and efficient static array bound checking for large embedded C programs". In: *Proceedings of the ACM SIGPLAN 2004 Conference on Programming Language Design and Implementation*. PLDI '04. Washington DC, USA: Association for Computing Machinery, 2004, pp. 231–242. ISBN: 1581138075. DOI: 10.1145/996841.996869. URL: https://doi.org/10.1145/996841.996869.

[33] Christoph Cullmann and Florian Martin. "Data-Flow Based Detection of Loop Bounds". In: *7th International Workshop on Worst-Case Execution Time Analysis (WCET'07)*. Ed. by Christine Rochange. Vol. 6. Open Access Series in Informatics (OASIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2007, pp. 1–6. ISBN: 978-3-939897-05-7. DOI: 10.4230/OASIcs.WCET.2007.1193. URL: https://drops.dagstuhl.de/entities/document/10.4230/OASIcs.WCET.2007.1193.

[34] Andreas Ermedahl et al. "Loop Bound Analysis based on a Combination of Program Slicing, Abstract Interpretation, and Invariant Analysis". In: *7th International Workshop on Worst-Case Execution Time Analysis (WCET'07)*. Ed. by Christine Rochange. Vol. 6. Open Access Series in Informatics (OASIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2007, pp. 1–6. ISBN: 978-3-939897-05-7. DOI: 10.4230/OASIcs.WCET.2007.1194. URL: https://drops.dagstuhl.de/entities/document/10.4230/OASIcs.WCET.2007.1194.

[35] Helmut Seidl, Reinhard Wilhelm, and Sebastian Hack. *Compiler design: Analysis and transformation*. English. Vol. 9783642175480. Publisher Copyright: © Springer-Verlag Berlin Heidelberg 2012. All rights are reserved. Springer-Verlag Berlin Heidelberg, Nov. 2013. ISBN: 3642175473. DOI: 10.1007/978-3-642-17548-0.

[36] Mohamed Abdel Maksoud and Jan Reineke. "A Compiler Optimization to Increase the Efficiency of WCET Analysis". In: *Proceedings of the 22nd International Conference on Real-Time Networks and Systems*. RTNS '14. Versaille, France: Association for Computing Machinery, 2014, pp. 87–96. ISBN: 9781450327275. DOI: 10.1145/2659787.2659825. URL: https://doi.org/10.1145/2659787.2659825.

[37] Benjamin Brosgol. "Do-178c: the next avionics safety standard". In: *Ada Lett.* 31.3 (Nov. 2011), pp. 5–6. ISSN: 1094-3641. DOI: 10.1145/2070336.2070341. URL: https://doi.org/10.1145/2070336.2070341.

Augsburg, November 1, 2025 Jonas Kell